23

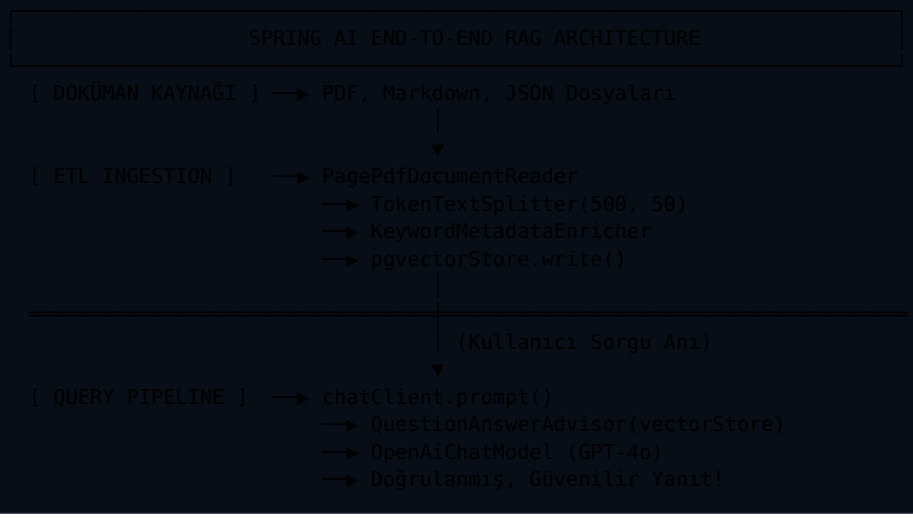
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Spring AI ile Uçtan Uca RAG Pipeline (Reader -> Transformer -> VectorStore)

End-to-End Spring AI RAG Pipeline: Ingestion, Transformation & Retrieval

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir fabrikanın üretim bandını hayal edin: Bandın başında içeri ham demir girer (PDF dokümanları), pres makineleri demiri parçalara böler (TokenTextSplitter), üzerine kalite damgası basılır (Metadata), ardından depodaki doğru raflara yerleştirilir (VectorStore). Müşteri bir parça sipariş ettiğinde ise sistem depodan en uygun parçayı şipşak çeker ve montajlayıp müşteriye teslim eder (ChatClient + Advisor)! Spring AI işte bu koca fabrikayı sadece birkaç Spring Bean'i ile sıfırdan kurmanızı sağlar!

🇬🇧 ENGLISH (ELI5)

Picture a high-efficiency manufacturing plant: raw steel enters the conveyor (PDFs), laser cutters slice it into uniform blocks (TokenTextSplitter), robotic arms stamp serial numbers and metadata, and automated cranes shelve them into vertical storage racks (VectorStore). When a custom order arrives, the system grabs matching parts and assemblies the finished product instantly (ChatClient + QuestionAnswerAdvisor). Spring AI encapsulates this entire factory in a few elegant Spring Beans!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Spring AI RAG Pipeline Mimarisi: 1) Ayrık Yaşam Döngüsü: Ingestion (Yükleme) pipeline'ı ile Query (Sorgulama) pipeline'ı birbirinden bağımsız çalışmalıdır. Ingestion arka plan batch job veya event-driven (Kafka/S3 Event) olarak çalışır. 2) `QuestionAnswerAdvisor`: Gelen sorguyu alır, embedding üretir, VectorStore'da benzerlik araması yapar, gelen dokümanları sistem prompt şablonundaki `{question_answer_context}` alanına yerleştirir. 3) Context Window Koruması: Advisor `order` sıralamasıyla Token sayım ve güvenlik filtrelerinin arkasına yerleştirilebilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Decoupled RAG Pipeline Architecture: 1) Temporal Decoupling: Ingestion (ETL) and Query servicing must never share execution lifecycles. Ingestion runs asynchronously via message queues (Kafka, S3 ObjectCreated events), while queries execute on low-latency HTTP paths. 2) `QuestionAnswerAdvisor` Mechanics: Intercepts user prompts, orchestrates similarity search against the vector store, formats retrieved segments into a markdown context block, and injects it into `{question_answer_context}`. 3) Advisor Ordering: Advisors must be prioritized to execute authorization before vector queries.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Üretim Seviyesinde Tam RAG Servis Mimarisi
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.stereotype.Service;

@Service
public class EnterpriseKnowledgeRagService {
    private final ChatClient chatClient;

    public EnterpriseKnowledgeRagService(ChatClient.Builder builder, VectorStore vectorStore) {
        var qaAdvisor = new QuestionAnswerAdvisor(
            vectorStore,
            SearchRequest.builder().topK(4).similarityThreshold(0.72).build(),
            "Aşağıdaki şirket bağlamını kullanarak soruyu yanıtla. Bilgi yoksa 'Bilmiyorum' de:
\n{question_answer_context}"
        );

        this.chatClient = builder
            .defaultSystem("Sen kurumsal bilgi yönetim asistanısın.")
            .defaultAdvisors(qaAdvisor)
            .build();
    }

    public String queryKnowledge(String userQuestion) {
        return chatClient.prompt()
            .user(userQuestion)
            .call()
            .content();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Darboğaz: Kullanıcı her soru sorduğunda dokümanı tekrar tekrar parse edip diske yazmaya çalışmayın! Ingestion pipeline'ı sadece doküman değiştiğinde veya yeni dosya yüklendiğinde bir kez çalışmalı; sorgu anında sadece hazır vektörler okunmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Architectural antipattern: Re-indexing documents on the fly during interactive query handling. Ingestion must remain strictly decoupled, executing asynchronously on document update triggers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Ingestion (Yükleme) ve Query (Sorgulama) pipeline'ları mutlaka zamansal olarak ayrışmalıdır.
- QuestionAnswerAdvisor, RAG arama ve bağlam enjeksiyonunu otomatikleştirir.
- Özel QA şablonu ile 'bilgi yoksa uydurma' kuralı Advisor seviyesinde garanti edilir.

🎯 Key Takeaways for the Interview

- Ingestion and query servicing must be decoupled into independent operational paths.
- QuestionAnswerAdvisor abstracts retrieval, threshold filtering, and context hydration.
- Custom QA templates enforce negative hallucination boundaries at the advisor layer.